

Gerenciador de Log de E- mails em C



Estrutura de Dados com Lista Encadeada

Equipe: Gabriel B, Fernando S e Victor F

Disciplina: Técnica de Programação

Linguagem: C (compilação com GCC)

Objetivo do Sistema

Sistema completo de cadastro, busca e persistência de registros de e-mail utilizando estrutura de dados dinâmica em C.

Funcionalidades Principais

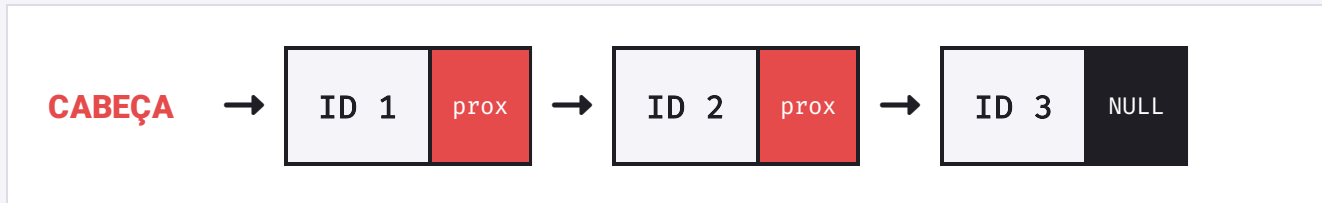
- ▶ Cadastro interativo de registros de e-mail com validação completa
- ▶ Armazenamento em memória via lista encadeada (alocação dinâmica)
- ▶ Busca por nome (substring, case-insensitive) e por data
- ▶ Remoção de registros por ID com ajuste automático de ponteiros
- ▶ Importação em lote a partir de arquivos de texto (.txt)
- ▶ Exportação e recuperação de dados em formato binário (.bin)

ESTRUTURA DO REGISTRO

```
typedef struct {  
    int id;  
    char nome[TAM_NOME];    // 100 caracteres  
    char email[TAM_EMAIL];  // 150 caracteres  
    char data[TAM_DATA];    // DD/MM/AAAA  
    char hora[TAM_HORA];    // HH:MM:SS  
} Registro;
```

Estrutura da Lista Encadeada

Representação Visual



Comportamento da Estrutura

- **Cada nó possui:** Um registro completo com todos os dados do e-mail e um ponteiro para o próximo nó.
- **Inserção:** Ocorre sempre no final da lista, garantindo a ordem cronológica de entrada.
- **Remoção:** Ajusta os ponteiros dinamicamente (o nó anterior passa a apontar para o próximo do nó removido).
- **Busca:** Percorre a lista sequencialmente do início (cabeça) até encontrar o elemento ou chegar ao fim (NULL).

Definição em C

```
/* Estrutura do Nó */
typedef struct No {
    Registro reg;
    struct No *prox;
} No;
```

```
/* Estrutura da Lista */
typedef struct {
    No *cabeca;
    int tamanho;
    int proximo_id;
} Lista;
```

Uso de malloc/free e Ponteiros

Alocação dinâmica garante uso eficiente de memória

+ Alocação (malloc)

criação de nó

```
No *criar_no(const Registro *r) {
    No *novo = (No *)malloc(sizeof(No));
    if (!novo) {
        fprintf(stderr, "ERRO: malloc falhou.\n");
        return NULL;
    }
    novo->reg = *r;
    novo->prox = NULL;
    return novo;
}
```

malloc cria nós dinamicamente apenas quando necessário, evitando o desperdício de memória de arrays estáticos. Os **ponteiros** (prox) conectam os nós formando a cadeia.

🗑️ Liberação (free)

LIBERAÇÃO NA REMOÇÃO

```
/* Após ajustar os ponteiros... */
free(cur);
l->tamanho--;
```

LIBERAÇÃO TOTAL DA LISTA

```
void liberar_lista(Lista *l) {
    No *cur = l->cabeca;
    while (cur) {
        No *tmp = cur->prox;
        free(cur);
        cur = tmp;
    }
}
```

free devolve a memória ao sistema operacional, prevenindo **vazamentos de memória (memory leaks)** ao remover um item ou encerrar o programa.

Inserção, Busca e Remoção

Três operações fundamentais que demonstram domínio de ponteiros e percurso de lista

- ▶ **Percurso completo** da lista para operações de inserção (encontrar o final) e busca.
- ▶ **Ajuste de ponteiros** na remoção: o nó anterior passa a apontar para o próximo do nó removido.
- ▶ **Caso especial:** a remoção do primeiro nó exige a atualização direta do ponteiro da cabeça da lista.

+ Inserção (no final)

```
if (!l->cabeca) {  
    l->cabeca = novo;  
} else {  
    No *cur = l->cabeca;  
    while (cur->prox) cur = cur->prox;  
    cur->prox = novo;  
}
```

Q Busca (por nome, case-insensitive)

```
if (strstr(nome_low, b)) {  
    exibir_registro(cur);  
    achou++;  
}
```

🗑 Remoção (por ID com ajuste de ponteiros)

```
if (!ant) {  
    l->cabeca = cur->prox;    // era o primeiro  
} else {  
    ant->prox = cur->prox;    // meio ou último  
}  
free(cur);
```

Tratamento de Erros e Validações

Validações robustas garantem integridade dos dados e experiência segura ao usuário

ERROS TRATADOS NO SISTEMA

- ⚠ **Data impossível:** Ex: 31/02/2026 ou dias fora do limite do mês.
- ⚠ **Horário inválido:** Horas > 23 ou minutos/segundos > 59.
- ⚠ **E-mail malformato:** Ausência de '@' ou domínio incompleto.
- ⚠ **Entrada vazia:** Campos obrigatórios não preenchidos.
- ⚠ **ID inválido:** Tentativa de remoção com ID inexistente ou negativo.
- ⚠ **Falha de arquivo:** Erro ao abrir arquivos para leitura ou escrita.

VALIDAÇÃO DE E-MAIL

```
const char *at = strchr(email, '@'); if (!at || at == email) return 0;
const char *ponto = strchr(at, '.'); if (!ponto || ponto == at + 1) return 0;
```

VALIDAÇÃO DE DATA (COM ANO BISSEXTO)

```
int dias_mes[] = {0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
if ((a % 4 == 0 && a % 100 != 0) || (a % 400 == 0))
    dias_mes[2] = 29;
if (d > dias_mes[m]) return 0;
```

VALIDAÇÃO DE HORA

```
if (h < 0 || h > 23) return 0;
if (m < 0 || m > 59) return 0;
if (s < 0 || s > 59) return 0;
```

Manipulação de Arquivos

Persistência de dados com importação de texto (.txt) e exportação/recuperação binária (.bin)

Arquivo Texto (.txt)

IMPORTAÇÃO

```
FILE *f = fopen(caminho, "r");  
// Formato: NOME: João; EMAIL: joao@gmail.com; ...
```

- ▶ **Importação em lote:** Suporta o carregamento de milhares de registros simultaneamente.
- ▶ **Parsing estruturado:** Extrai dados formatados campo a campo (NOME, EMAIL, DATA, HORA).
- ▶ **Validação rigorosa:** Cada registro é validado individualmente antes de ser inserido na lista encadeada.

Arquivo Binário (.bin)

EXPORTAÇÃO E RECUPERAÇÃO

```
// Exportação  
fwrite(&l->proximo_id, sizeof(int), 1, f);  
fwrite(&cur->reg, sizeof(Registro), 1, f);  
  
// Recuperação  
fread(&pid, sizeof(int), 1, f);  
fread(&r, sizeof(Registro), 1, f);
```

- ▶ **Alta performance:** Leitura e escrita direta em memória, sendo significativamente mais rápido que arquivos de texto.
- ▶ **Preservação estrutural:** Mantém a estrutura exata dos dados (struct Registro) sem necessidade de parsing.
- ▶ **Recuperação de estado:** Permite restaurar todos os registros e o contador de IDs ao reabrir o programa.

Modularização do Código

Código organizado em funções específicas facilita manutenção e reutilização

FUNÇÃO	RESPONSABILIDADE
<code>menu_adicionar()</code>	Interface de cadastro interativo
<code>procurar_nome()</code>	Busca por substring no nome
<code>procurar_data()</code>	Busca por data exata
<code>validar_data()</code>	Validação de formato e existência da data
<code>validar_email()</code>	Validação de formato de e-mail
<code>exportar_binario()</code>	Salva lista em arquivo .bin
<code>recuperar_binario()</code>	Restaura lista a partir de .bin
<code>liberar_lista()</code>	Desaloca toda a memória

BENEFÍCIOS

- ✓ **Código organizado e legível**
613 linhas bem estruturadas e divididas logicamente.
- ✓ **Fácil manutenção e depuração**
Erros podem ser isolados em funções específicas sem afetar o resto do sistema.
- ✓ **Reutilização de funções**
Exemplo: `criar_no()` é utilizado tanto na inserção quanto na recuperação de dados.

Demonstração Prática

Fluxo completo de demonstração no terminal comprova o funcionamento do sistema

01 ADICIONAR

Mostrar validação funcionando (testar e-mail inválido, data impossível como 31/02/2026).

02 LISTAR

Exibir todos os registros cadastrados com formatação tabular.

03 BUSCAR

Buscar por nome parcial (case-insensitive).

04 REMOVER

Remover um registro por ID e confirmar a remoção listando novamente.

05 IMPORTAR .TXT

Carregar o arquivo log_emails.txt com milhares de registros.

06 EXPORTAR .BIN

Gerar o arquivo emails.bin com os dados em memória.

07 RECUPERAR .BIN

Fechar o programa, reabrir e recuperar todos os dados do arquivo binário instantaneamente.

Isso prova na prática:

- ✓ **Persistência de dados**
- ✓ **Lista dinâmica funcional**
- ✓ **Manipulação correta de arquivos**